



Creating a numerical physical model of a water rocket and comparing it with experimental results

Abschließende Arbeit verfasst von
Foma Skorokhod
Klasse 8B

Betreuer: Alexander Socher

12.2.2026

Gymnasium und Realgymnasium Wien 3
Radetzkystraße 2a, 1030 Wien

Table of Contents

Abstract:	3
Introduction:.....	3
Chapter 1: Research methodology	3
1.1. Physical principles	3
1.2. Software used.....	5
1.3. Overview of the data comparison methods	6
Chapter 2: The description of the numerical model	6
2.1. Overview of the model	6
2.2. Expressing the equation for $u(t)$	8
2.3. Calculating $a(t)$ and $v(t)$ from $u(t)$	9
2.4. Solution methods.....	9
2.5. Implementing the solution	10
2.6. Picking parameters.....	11
Estimating A_c :.....	12
Estimating ρ_a :	12
Estimating C_D :.....	13
Chapter 3: The experimental setup.....	14
3.1. Measuring equipment.....	14
3.2. The rocket.....	15
3.3. The launchpad	16
3.4. The launch and the measurements.....	16
Chapter 4: Data comparison and analysis.....	17
Chapter 5: Discussion of results	20
5.1. Forming hypotheses	20
5.2. The error due to adiabatic expansion assumption	20
5.3. The effect of the hydrostatic pressure:	22
5.4. Evaluating thrust produced by air leaving the bottle	23
5.5. The error due to the solid-body assumption	24
Conclusion:.....	25
Acknowledgments:	25
Bibliography:	26

Appendix 1: Notes on creating the numerical model	28
Appendix 2: Notes on creating the launchpad and the rocket	29
Appendix 3: Notes on creating the measurement system	31

Abstract:

This paper describes the process of creating a simplified numerical model of a water rocket as well as an experimental setup to measure the maximum altitude during the rocket's flight to compare it to the model's prediction. The data comparison shows that the created model does not produce accurate results due to simplifications made. Those simplifications include modelling the air expansion as an adiabatic process, neglecting the reactive thrust produced by air expelled from the rocket and treating the rocket containing water as a single solid body.

Introduction:

A water rocket is a relatively simple physics experiment. Water and pressurised air inside a plastic bottle act as a thruster, enabling the rocket to reach altitudes exceeding 50 m. Air expands and expels water through the nozzle, creating reactive thrust. This experiment is known to nearly every science-interested school student worldwide. The goal of this study is therefore, to determine how accurate a numerical model of a water rocket based on high-school-level physics can be. A water rocket with a launchpad and on-board accelerometer and barometer must be designed and assembled to compare the maximum altitude reached by the rocket with the values predicted by the model.

Chapter 1: Research methodology

This chapter provides an overview of the tools and methods that will be used throughout the paper to give readers a general understanding of the terminology and techniques employed. All of the topics covered in this chapter will be explained in greater detail later.

1.1. Physical principles

The thrust of a water rocket is produced mainly by water exiting the rocket through the nozzle as it is being pushed by the expanding air. Although air leaving the rocket after all

the water has escaped also produces thrust, the water remains the primary source of momentum. To calculate the thrust of the rocket produced by the water, the reactive thrust equation in the following form is used¹:

$$T = u \frac{dm}{dt} \quad (1)$$

where T is thrust, t is time, u is the velocity of water leaving the rocket and m is the mass of the rocket.

To express the equation for $u(t)$, Bernoulli's equation in the following form² is used.

$$p - p_a = \frac{\rho u^2}{2} \quad (2)$$

p is the water pressure inside the rocket, p_a is the atmospheric pressure, ρ is the water density. The Bernoulli's equation used here implies that the water flow is incompressible, which is not violated at the pressures used in this project.³ This form of Bernoulli's equation also does not account for energy losses due to turbulence.

The effect of hydrostatic pressure⁴ $p_{hs} = \rho gh$ inside the rocket is minimal⁵, therefore, the water pressure and the air pressure inside the rocket are assumed to be equal.

As air expands and pushes water out of the rocket, its pressure and therefore the pressure of the water decrease, causing u to reduce as well. Within the scope of this work, air is assumed to be an ideal gas that undergoes an adiabatic expansion, described by the following equation⁶, as it allows linking pressure to volume.

$$p_0 V_0^\gamma = p V^\gamma = C \quad (3)$$

p_0 and V_0 are the initial air pressure and air volume inside the rocket. γ is the heat capacity ratio, p and V are the air pressure and volume at any given time, C is a constant. Due to water evaporating inside the rocket during the flight and the flight preparation phase, the adiabatic coefficient γ may not remain constant. This will not be accounted for. Instead, γ for dry air will be used.

¹ Yuqi Yang et al., "Study on Thrust Performance of Small Water Rocket Launch," *Journal of Physics: Conference Series* 2313, no. 1 (2022): 4, <https://doi.org/10.1088/1742-6596/2313/1/012021>.

² <https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/bernoullis-equation/>

³ Yang et al., "Study on Thrust Performance of Small Water Rocket Launch," 4.

⁴ Roberto Mauri, *Transport Phenomena in Multiphase Flows*, Fluid Mechanics and Its Applications 112 (Springer, 2015), 112, <https://doi.org/10.1007/978-3-319-15793-1>.

⁵ See Section 5.3 for evaluation

⁶ Graham Woan, *The Cambridge Handbook of Physics Formulas*, 1st ed. (Cambridge University Press, 2000), 110, <https://doi.org/10.1017/CBO9780511755828>.

Bernoulli's equation and the adiabatic process equation from this chapter are used to form a differential equation for $u(t)$, cannot be solved analytically, with tools employed in this work. Therefore, a physical model must utilize numerical methods.

1.2. Software used

This subchapter provides an overview of the software and online services employed in this work.

Wolfram Engine⁷ – CAS software used for symbolic computation, solving differential equations analytically and estimating the air drag coefficient of the rocket. It was chosen for its functionality and its free noncommercial license.

Autodesk Fusion⁸ – CAD software used for designing the rocket and the launchpad, creating rendered images and estimating the cross-sectional area of the rocket. It was chosen for ease of use and its free noncommercial license.

Arduino IDE⁹ – development environment used to create the firmware for the on-board sensor system. It was chosen because of its ease of use, prior experience with the platform and good support for the sensors used.

Microsoft Excel¹⁰ – used for data comparison and results visualization

GitHub¹¹ – online platform used for publishing the source code of the numerical model and of the measurement system's firmware.

Python¹² – programming language powering the numerical model. It was chosen for its ease of use and the availability of libraries

NumPy¹³ – Python library used to optimize array operations

Matplotlib¹⁴ – Python library used to visualize the output of the model

MetPy¹⁵ – Python library used to estimate air density on the launch site

Open-Meteo¹⁶ – online meteorological service, source for air temperature and humidity on the launch site.

⁷ *Wolfram Engine*, n.d., <https://www.wolfram.com/engine/>.

⁸ *Autodesk Fusion*, n.d., <https://www.autodesk.com/eu/products/fusion-360/overview>.

⁹ *Arduino IDE*, n.d., <https://www.arduino.cc/en/software/>.

¹⁰ *Microsoft Excel*, n.d., <https://excel.cloud.microsoft/en-us/>.

¹¹ *GitHub*, n.d., <https://github.com/>.

¹² *Python*, v. 3.12.6, n.d., <https://www.python.org/>.

¹³ *NumPy*, n.d., <https://numpy.org/>.

¹⁴ *Matplotlib*, n.d., <https://matplotlib.org/>.

¹⁵ *MetPy*, n.d., <https://unidata.github.io/MetPy/latest/index.html>.

¹⁶ *Open-Meteo*, n.d., <https://open-meteo.com/>.

1.3. Overview of the data comparison methods

The peak altitudes of the rocket, recorded by the on-board barometer, as well as the values predicted by the model will be imported into Microsoft Excel. Then, statistical methods such as mean bias error, relative error, and correlation coefficient are used to evaluate the model's performance. Based on these results, possible reasons for discrepancies are discussed in Chapter 5.

Chapter 2: The description of the numerical model

This chapter provides a detailed description of the process of deriving the differential equations that can be solved for acceleration $a(t)$ or velocity $v(t)$ and solving them numerically with Python.

2.1. Overview of the model

The term “numerical model” is used to describe a tool that can numerically solve differential equations, which determine the flight parameters of the rocket. The differential equations are based on Bernoulli's equation - Eq. (2) to estimate the velocity of water leaving the rocket. This is necessary in order to later use the Eq. (1) to calculate the thrust of the rocket, as well as the mass of the water remaining in the rocket¹⁷.

By knowing the initial air volume inside the rocket V_0 , having the function $u(t)$ or its discrete values and knowing the nozzle diameter, the function describing the water volume inside the rocket can be approximated.

However, the air drag¹⁸ - $F_D = \frac{1}{2} C_D \rho_a A_c v^2$ must also be taken into account. It affects $a(t)$ and depends on the velocity of the rocket and drag coefficient C_D , air density ρ_a , cross-sectional area of the rocket A_c and velocity v . More differential equations are required¹⁹.

After $u(t)$ is calculated, any further calculation will be based on it. Therefore, it is necessary to calculate $u(t)$ as precisely as possible, to avoid the accumulation of integration errors. In general, when finding numerical solutions, Runge-Kutta methods

¹⁷ See the next subchapter and <https://real-world-physics-problems.com/water-rocket-physics.html>

¹⁸ *Drag Equation*, n.d., <https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/drag-equation/>.

¹⁹ See Sections 2.2 and 2.3 and <https://real-world-physics-problems.com/water-rocket-physics.html>

are a good option²⁰. For $u(t)$, the more precise RK4 method will be used. It provides better accuracy than RK1 (Euler's Method) but does not overcomplicate the model²¹. RK4 will allow for the calculation of discrete values of $u(t)$ with the time step h . The reason for not using RK4 for further calculations is that while RK4 returns values with the step h , it needs values that lie between the values of step h [²²]. So, either the time step must be increased or a simpler method such as RK1 - which does not require intermediate values - will have to be used. The latter option was chosen, as it avoids increasing the model's complexity while maintaining decent accuracy.

To implement the Runge-Kutta (RK) methods and solve the differential equations, a Python script using NumPy is created. Python offers simple syntax, while being capable of fulfilling all requirements of this numerical model, thanks to a wide range of libraries such as NumPy or Matplotlib. Matplotlib will also be used in the model for visualizing the output.

Moreover, the model does not include the momentum created by the air leaving the rocket after the water has fully exited.

²⁰ J. R. Dormand and P. J. Prince, "A Family of Embedded Runge-Kutta Formulae," *Journal of Computational and Applied Mathematics* 6, no. 1 (1980): 1, [https://doi.org/10.1016/0771-050X\(80\)90013-3](https://doi.org/10.1016/0771-050X(80)90013-3).

²¹ Department of Physics, Shivaji College (University of Delhi), Raja Garden New Delhi-110027 et al., "Numerical Approximation Methods and Comparison with RK-4 Method for a Linear Differential Equation with Initial Conditions Using Scilab 6.1.1," *Current Natural Sciences and Engineering* 1, no. 6 (2024): 471, <https://doi.org/10.63015/5c-2447.1.6>.

²² Department of Physics, Shivaji College (University of Delhi), Raja Garden New Delhi-110027 et al., "Numerical Approximation Methods and Comparison with RK-4 Method for a Linear Differential Equation with Initial Conditions Using Scilab 6.1.1," 472.

2.2. Expressing the equation for $u(t)$

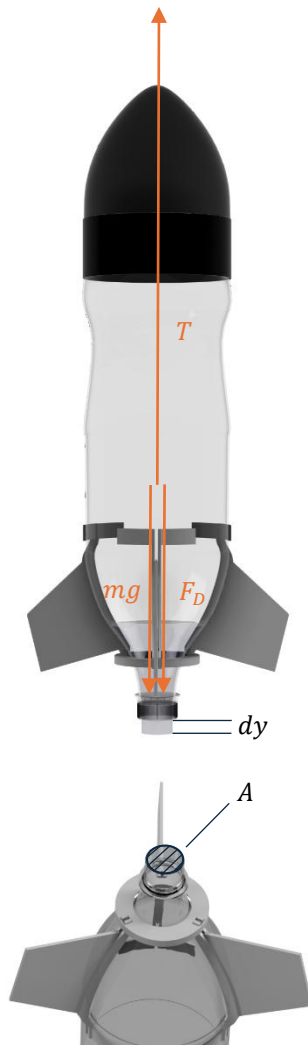


Figure 1: Forces during ascent and $u = dy/dt$ diagrams. h or y axes is directed upward

$$C = p_0 V_0^\gamma = p V^\gamma \quad (4)$$

$$p = \frac{C}{V^\gamma} \quad (5)$$

By substituting (5) into (2):

$$\frac{C}{V^\gamma} - p_{atm} = \frac{\rho u^2}{2} \quad (6)$$

After differentiating both sides of (6):

$$\frac{-C\gamma}{V^{\gamma+1}} \cdot \frac{dV}{dt} = \rho u \cdot \frac{du}{dt} \quad (7)$$

Assuming that the water exiting the nozzle of area A travels distance dy in a time interval dt , as illustrated in Fig. 1, the volume of the water escaping the rocket in dt seconds is $dV = dy \cdot A$, because²³ $V = y\pi r^2 = yA$.

Since $\frac{dy}{dt} = u$ by definition²⁴, the rate of change of the water volume in the rocket can be expressed in the following way:

$$\frac{dV}{dt} = \frac{dy}{dt} \cdot A = u \cdot A \quad (8)$$

After substituting (8) into (7):

$$\frac{-C\gamma}{V^{\gamma+1}} \cdot u \cdot A = \rho u \cdot \frac{du}{dt} \quad (9)$$

Which simplifies to:

$$\frac{du}{dt} = \frac{-C\gamma}{V^{\gamma+1}} \cdot \frac{A}{\rho} \quad (10)$$

From (6), V can be expressed as:

$$V = \left(\frac{2C}{2p_{atm} + \rho u^2} \right)^{\frac{1}{\gamma}} \quad (11)$$

By substituting V in (10) with (11) the differential equation of $u(t)$ in explicit form is obtained:

²³ Woan, *The Cambridge Handbook of Physics Formulas*, 37.

²⁴ Paul Allen Tipler and Gene Mosca, *Physik: für Wissenschaftler und Ingenieure*, 7. dt. Aufl, Lehrbuch (Springer Spektrum, 2015), 30.

$$\frac{du}{dt} = -C\gamma \cdot \frac{A}{\rho} \cdot \left(\frac{2C}{2p_{atm} + \rho u^2} \right)^{\frac{-\gamma-1}{\gamma}} \quad (12)$$

2.3. Calculating $a(t)$ and $v(t)$ from $u(t)$

Calculating the function of the rocket's velocity over time – $v(t)$ is the objective of this section, since the rocket's altitude as a function of time $h(t)$ is the integral of $v(t)$.

Thrust, gravity, and air drag are the only substantial forces acting on the rocket. The acceleration of the rocket is the sum of those three divided by the rocket's mass.

$$\text{Since}^{25} m = \rho \cdot V \quad (13)$$

$$\frac{dm}{dt} = \rho \cdot \frac{dV}{dt} = \rho \cdot A \frac{dy}{dt} = \rho u A \quad (14)$$

This implies that the thrust can be expressed as:

$$T = \rho u^2 A \quad (15)$$

By using (8) and (13), $m(t)$ can be expressed as:

$$m(t) = m_0 - \int \rho u A \cdot dt \quad (16)$$

From Newton's second law ($a = F/m$)²⁶, and as illustrated in Fig. 1:

$$a = \frac{T - F_D(v) - mg}{m} = \frac{dv}{dt} \text{ (during ascent)} \quad (17)$$

where v is the velocity. This yields a differential equation for $v(t)$.

2.4. Solution methods

The attempts to solve $u(t)$ analytically using the Wolfram Engine were made but did not lead to an explicit solution. Therefore, numerical methods were chosen as a more reliable approach.

RK1 (Euler's method) and RK4 are used in the following forms:

Euler's method (RK1)²⁷ :

²⁵ Tipler and Mosca, *Physik*, 372.

²⁶ Woan, *The Cambridge Handbook of Physics Formulas*, 68.

²⁷ Woan, *The Cambridge Handbook of Physics Formulas*, 62.

$$\frac{dy}{dx} = f(x, y)$$

$$x_{n+1} = x_n + h$$

$$y_{n+1} = y_n + hf(x_n, y_n)$$

Runge-Kutta method (RK4)²⁸:

$$\frac{dy}{dx} = f(x, y)$$

$$x_{n+1} = x_n + h,$$

$$k_1 = hf(x_n, y_n)$$

$$k_2 = hf(x_n + h/2, y_n + k_1/2)$$

$$k_3 = hf(x_n + h/2, y_n + k_2/2)$$

$$k_4 = hf(x_n + h, y_n + k_3)$$

$$y_{n+1} = y_n + \frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6}$$

Where h is the step size.

2.5. Implementing the solution

Since later calculations are based on the solution for Eq. (12), it is necessary to solve it as precisely as possible, therefore RK4 is employed. For the reasons stated in Chapter 2, Eq. (17) is solved using RK1, which results in discrete values of $v(t)$.

While there is no difficulty in implementing methods and equations described above as described in the first chapter, some considerations must be taken. If statements must be implemented while calculating $u(t)$ or $m(t)$ and $T(t)$, to discard the $u(t)$ values that would appear after all the water has left the rocket. Otherwise, all calculations would continue as normal, while the rocket would continue producing thrust without water and would obtain a negative mass as seen in Fig. 2, due to $m(t)$ being calculated from $u(t)$.

²⁸ Woan, *The Cambridge Handbook of Physics Formulas*, 62.

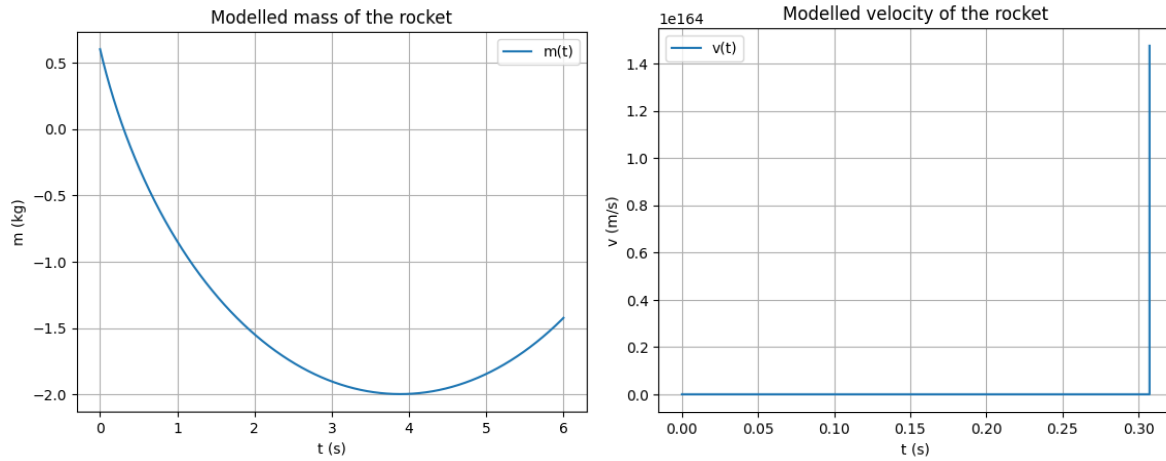


Figure 2: Rocket obtaining negative mass and exceeding the speed of light without emptiness checks with the parameters similar to default ones.

In the Python script used throughout this paper $m(t)$ has checks to prevent it from falling below the mass of the empty rocket and $T(t)$ has checks to set it to 0 when the rocket runs out of water.

After $v(t)$ is calculated, obtaining the maximum altitude requires numerically integrating $v(t)$ and finding the maximum value using the NumPy function²⁹ `numpy.max()`.

The full script can be found on GitHub³⁰.

See Appendix 1 for more information on the model

2.6. Picking parameters

For the constants, the following values were chosen (Table 1):

Table 1: Constants

Constant	Designation	Value
Heat capacity ratio	γ	1.4
Water density	ρ	1000 kg/m^3
Atmospheric pressure	p_{atm}	99323 Pa and 100900 Pa
Cross sectional area of the rocket	A_c	0.007648398 m^2

²⁹ Numpy.Max, n.d., <https://numpy.org/doc/stable/reference/generated/numpy.max.html>.

³⁰ Foma Skorokhod, *Physical Numerical Model of a Water Rocket*, n.d., <https://github.com/tom-737/Physical-numerical-model-of-a-water-rocket>.

Nozzle area	A	$0.0003801327m^2$
Bottle volume	V_B	$0.0015m^3$
Gravitational constant	g	$9.81m/s^2$
Drag coefficient of the rocket	C_D	0.497405
Air density	ρ_a	$1.2694842991kg/m^3$ and $1.2790894396 kg/m^3$
Mass of the empty rocket	m_r	181 g

The following model parameters and rocket configurations will be referred to as default (Table 2):

Table 2: Default parameters

Initial water volume	V_w	500 ml
Initial air pressure in the rocket	p_0	$p_{atm} + 400 kPa$

Estimating A_c :

A projection of the rocket's 3D model was drawn onto a sketch perpendicular to the rocket's vertical axis. The area of the projection was displayed by Autodesk Fusion.

Estimating ρ_a :

Before launching the rocket, air humidity and temperature at the site were retrieved from Open-Meteo.

On the first day of launches, the temperature was $-1^\circ C$ at 71% relative humidity. Ground pressure was recorded as 993.23 hPa by the rocket's on-board barometer.

To calculate the air density under the described conditions, the `mixing_ratio_from_relative_humidity`³¹ and `density`³² functions of the MetPy Python library were used.

Launches at 400 *kPa* gauge pressure with 500 *ml* and 300 *ml* of water were conducted under the weather conditions described above. Other measurements were done later, under 1009 *hPa* atmospheric pressure, +1°C air temperature and 98% relative humidity.

Estimating C_D :

After the rocket reaches its maximum height and starts descending, it has no thrust, mass is constant and F_D is directed upward. Therefore, rocket's acceleration function changes from Eq. (17) to following form (F_D is expanded):

$$a(t) = -g + \frac{A_c C_D \rho_a v(t)^2}{2m_r}$$

Since a is dependent on v it is yet another differential equation.

Solving it using the Wolfram Engine yields:

$$v(t) = \frac{1.4121 \sqrt{g m_r}}{\sqrt{A_c C_D \rho_a}} \cdot \tanh\left(\frac{0.707107 \sqrt{A_c C_D g \rho_a} \cdot t}{\sqrt{m_r}}\right)$$

After integrating over t :

$$h(t) = h_0 - \frac{m_r \cdot \log\left(\sinh^2\left(\frac{0.707107 \sqrt{A_c C_D g \rho_a} \cdot t}{\sqrt{m_r}}\right) + 1\right)}{A_c C_D \rho_a}$$

A_c , g , ρ_a and m_r are known constants, and the $h(t)$ time series was measured by the on-board barometer. This makes it possible to approximate C_D using the `FindFit`³³ function of the Wolfram Engine.

$$C_D \approx 0.497405$$

$h_0 = 51.93$ *m*, even though the maximum height for this particular launch was recorded to be 52.02 *m*, which was achieved approximately 200 *ms* earlier. Both values lie within the error (± 0.66 *m*)³⁴ of the barometer used to measure the altitude. However, when using $h(t)$ values starting from 52.02 *m*, the “plateau” in the left side of Fig. 4 becomes too long for the symbolic function to fit the flight data. This gives $C_D \approx 1.16696$, and an $h(t)$ approximation

³¹ `Mixing_ratio_from_relative_humidity`, n.d.,

https://unidata.github.io/MetPy/latest/api/generated/metpy.calc.mixing_ratio_from_relative_humidity.html.

³² `Density`, n.d., <https://unidata.github.io/MetPy/latest/api/generated/metpy.calc.density.html>.

³³ <https://reference.wolfram.com/language/ref/FindFit.html>

³⁴ “BMP388 - Datasheet,” n.d., 3, <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp388-ds001.pdf>.

that differs significantly from the flight data. Therefore, 52.02 m may have been recorded during the ascent phase of the flight due to measurement uncertainty. Using $C_D \approx 0.497405$ in symbolic $h(t)$ solution for the descent results in a function that is very similar to the recorded timeseries, as illustrated in Fig. 5.

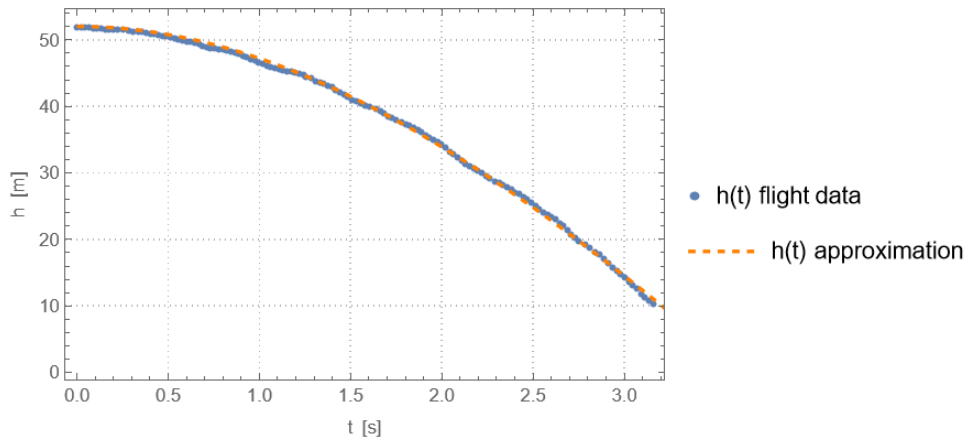


Figure 3: $h(t)$ with approximated drag coefficient compared to experimental data.

Chapter 3: The experimental setup

3.1. Measuring equipment



Figure 4: Measurement system V3

The measurement system is mainly focused on recording the rocket's altitude, as it will be used in data comparison, although acceleration is recorded as well.

Measurements of acceleration $a(t)$ and altitude $h(t)$ will be taken with respect to the setup parameters. These include the rocket's initial air pressure and water volume.

To measure the $a(t)$ time series, the Adafruit ADXL375³⁵ accelerometer was used. The ADXL375 was chosen because it has a wide measurement range. Adafruit BMP388³⁶

barometer was used for measuring the air pressure and

therefore the height of the rocket. It provides sufficient accuracy and has an I²C interface as does the accelerometer. Adafruit provides easy-to-use libraries^{37,38} for

³⁵ Adafruit ADXL375, n.d., <https://learn.adafruit.com/adafruit-adxl375>.

³⁶ Adafruit BMP388 and BMP390 - Precision Barometric Pressure and Altimeter, n.d., <https://learn.adafruit.com/adafruit-bmp388-bmp390-bmp3xx/overview>.

³⁷ Adafruit ADXL375 Accelerometer Driver, n.d., https://github.com/adafruit/Adafruit_ADXL375.

³⁸ Adafruit_BMP3XX, n.d., https://github.com/adafruit/Adafruit_BMP3XX.

Arduino IDE, making it straightforward to connect all sensors via I₂C to the ESP32 development board, and to program the system using Arduino IDE. The LOLIN32 Lite ESP32 development board was chosen because it has built-in circuitry to allow connecting a lithium cell directly, substantially simplifying the on-board electronics. The final version of the measurement system is pictured in Fig. 4.

Measurements will be taken throughout the flight and then be saved into a .csv file on a connected micro-SD card. The .csv file can later be imported into Microsoft Excel for further analysis. The measurements start after the system detects upward acceleration.

The code for the ESP32 development board can be found on GitHub³⁹.

See Appendix 3 for more information about the measurement system.

3.2. The rocket

The rocket is based on a 1.5L PET Coca-Cola bottle, as the popularity of the drink simplifies the search for a 3D-mesh of the bottle. The opening of the bottle functions as a nozzle. To mount the sensors on a bottle, a nosecone with proper mountings was designed in Autodesk Fusion and then 3D-printed. No parachute is used, as it allows the collection of data about the free fall of the rocket to estimate the C_D . Furthermore, stabilizers were 3D-printed to ensure that the rocket flew directly upward. The nozzle diameter is not changeable, as the only realistic way to implement it would be launching the rocket with the hole in its cap acting as a nozzle, allowing to switch between caps with different diameters. Doing so would make an orifice, which has non-negligible drag⁴⁰, that Bernoulli's equation in its used form (*Eq. (2)*) does not account for. Moreover, in many hobbyist water rockets, the nozzle diameter is not a definable parameter, due to the limited choice of off-the-shelf parts. The rocket is shown in Fig.5.



Figure 5: Final Version of the rocket

³⁹ Foma Skorokhod, *Water Rocket On-Board Altitude and Acceleration Logger*, n.d., <https://github.com/tom-737/Water-rocket-on-board-altitude-and-acceleration-logger>.

⁴⁰ S. Shaaban, "Optimization of Orifice Meter's Energy Consumption," *Chemical Engineering Research and Design* 92, no. 6 (2014): 1–2, <https://doi.org/10.1016/j.cherd.2013.08.022>.

See Appendix 2 for the description of the process of creating the rocket.

3.3. The launchpad

The launchpad must have a release mechanism that allows the launch at a specific time and pressure and maintains the air pressure inside the rocket before the launch. To achieve that, O-rings are used as seals at the bottle mount, while the zip-ties are used to implement the release mechanism. A circle of zip-ties will hold the bottle with their locks, while being prevented from bending out by a tube-shaped 3D-printed slider. The slider is locked in place by a metal pin pierced through it and the body of the launchpad, as shown in Fig. 7. When the pin is pulled out, the slider is pulled downward by the rubber bands, as demonstrated in Fig. 6. When the slider is lowered, the upper parts of zip-ties can be bent freely, allowing them to slip from the rocket and release it. A bicycle pump equipped with a pressure gauge is used to pressurize the bottle, as shown in Fig.8.

See Appendix 2 for more information on the launchpad.



Figure 7: Launchpad - released

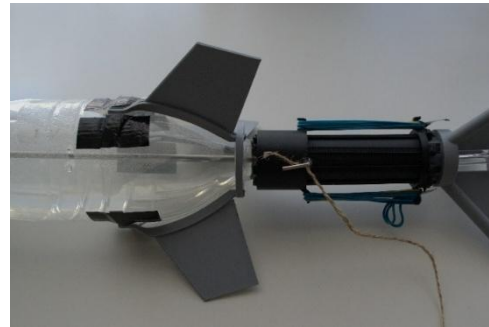


Figure 6: Launchpad - locked

3.4. The launch and the measurements

First, the bottle is filled with a specific amount of water and installed on the launchpad, as demonstrated in Fig.9. The Bluetooth signal is sent to the ESP32, initiating a waiting cycle until upward acceleration triggers the measurements. After the bottle is pressurized to a desired value, the pin is pulled out and the rocket is released. The sensors will record their values with the sample rate of approximately 50 Hz for 20 seconds. The flight data can then be exported and analyzed. A total of five measurement launches were conducted under different parameters, varying both the pressure and the water volume inside the rocket.



Figure 9: Launch site



Figure 8: Rocket on the launchpad

Chapter 4: Data comparison and analysis

This chapter compares the maximum predicted altitude of the model presented in this paper to the experimental data and two other water rocket simulators.

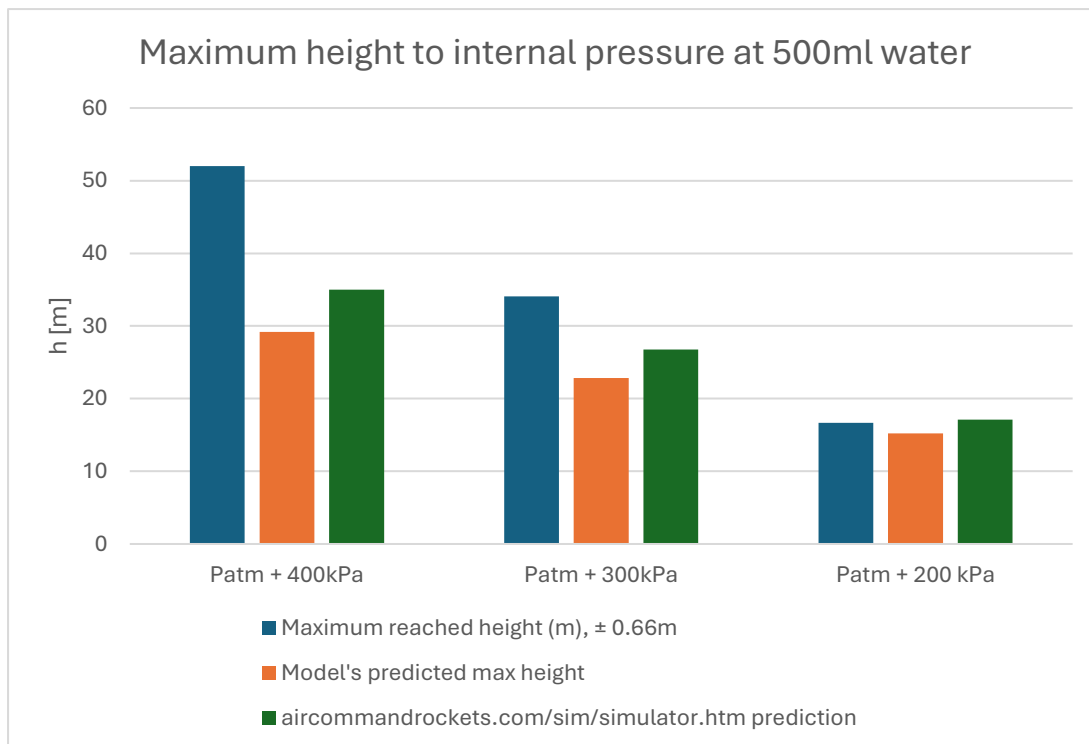


Figure 10: Maximum height comparison with air pressure as a variable parameter.

As seen in Fig. 10, the maximum height difference decreases significantly with pressure. The rocket becomes less stable under lower pressures. The reason for similar results is therefore not the accuracy of both models, but the rocket's non-vertical trajectory. This effect is, however, not noticeable during launches with 400 kPa gauge pressure.

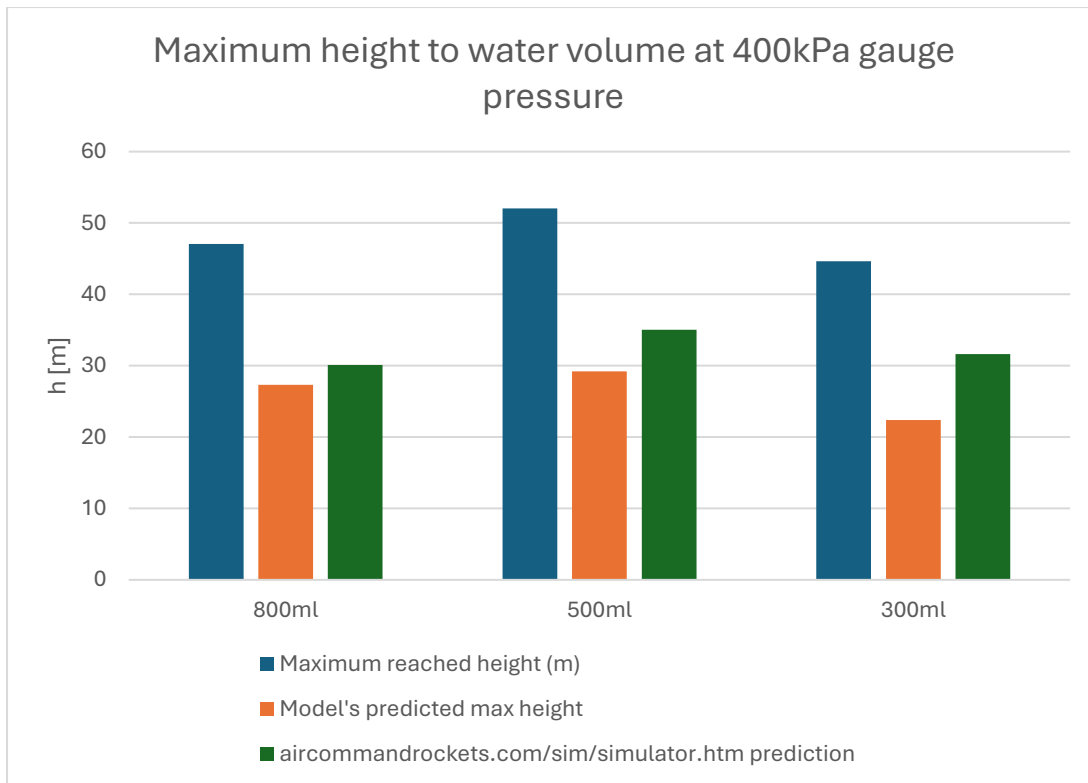


Figure 11: Maximum height comparison with water volume as a variable parameter.

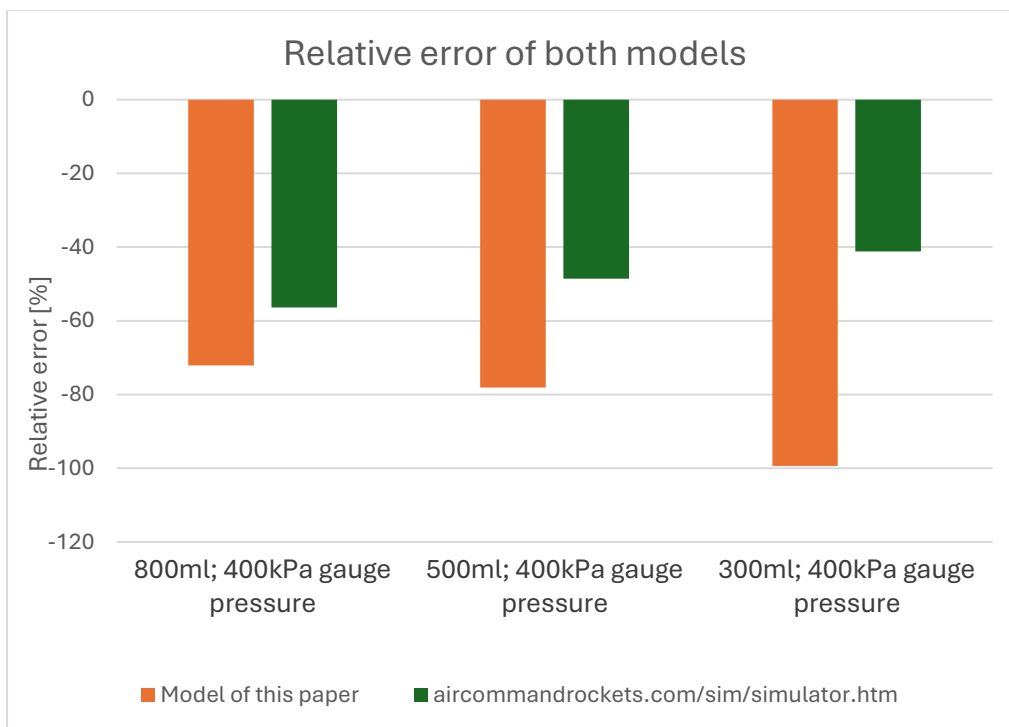


Figure 12: Relative error comparison.

The error of this paper's model increases when the rocket has less water, while the model from aircommandrockets.com has the opposite effect as can be seen in Fig. 12,

possibly because the latter model accounts for air thrust⁴¹, and less water means more pressurised air, hence more work that can be performed by air thrust.

Table 3: Correlation coefficients and mean bias errors of predicted maximum altitudes

Model:	Correlation coefficient	Mean bias error
This paper's model:	0.9	−21.6 m
aircommandrockets.com:	0.81	−15.7 m

Figs. 10-12 show how inaccurate both water rocket models are, but one statistical indicator stands out. The predictions of both models have surprisingly high correlation coefficients with the experimental data, as shown in Table 3, although coefficients are based on only three data points and therefore their representativeness is limited.

The Rocket Modeler-III from NASA⁴² performs differently, as can be seen in Table 4. It was not included in this comparison because it does not have an option for a 1.5 L bottle. Nevertheless, when using 2 L as bottle volume on all three models, with other parameters being as similar to previously defined ones as possible, a comparison can be made.

The following parameters are used for the next comparison:

$$C_d = 0.814 \quad m_r = 0.181 \text{ kg} \quad p_0 \approx 400 \text{ kPa} + p_{atm} \quad T_{air} = 25^\circ\text{C}$$

$$A_c = \pi 0.05^2 \text{ m}^2 \quad A = \pi 0.011^2 \text{ m}^2 \quad V_{w0} = 500 \text{ ml} \quad p_{atm} = 101.3 \text{ kPa}$$

Relative humidity is set to 100%. All other parameters are the same as in Section 2.6.

Table 4: Water rocket simulators comparison

Model:	Prediction:
Rocket Modeler-III	59 m
Simulator from aircommandrockets.com	35.17 m
Model presented in this paper	27.91 m

⁴¹ *Water Rocket Simulator*, n.d., <http://www.aircommandrockets.com/sim/simulator.htm>.

⁴² *Rocket Modeler-III*, n.d., <https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/rocketmodeler/>.

Chapter 5: Discussion of results

This chapter describes possible reasons for measured discrepancies and evaluates their effect on model's predictions.

5.1. Forming hypotheses

The most obvious starting point would be to challenge the simplifications made earlier. This leads to:

- The air expansion could be a non-adiabatic process.
- The hydrostatic pressure may be non-negligible.
- Thrust produced by air may be substantial.

There is, however, one more simplification, which was not made intentionally. *Eq. (17)* treats the rocket, with remaining water inside, as a single solid body.

- The rocket with water cannot be treated as a solid body.

5.2. The error due to adiabatic expansion assumption

An adiabatic expansion requires perfect thermal isolation, which the PET bottle does not have.

The ideal gas law in the following form⁴³ will be applied:

$$p \cdot V = nRT \quad (18)$$

where p and V are air pressure and volume, n is the amount of air, R is a constant and T is temperature.

The heat conduction equation in the following form⁴⁴ will be used:

$$\frac{\Delta Q}{\Delta t} = kA \frac{\Delta T}{\Delta x} \quad (19)$$

Where ΔQ is the transferred heat, Δt is time, A is the area of the heat exchange, Δx is the wall thickness of the bottle, ΔT is the temperature difference and k is the thermal conductivity constant.

In order for the adiabatic coefficient to contribute to the increase in the rocket's maximum height, the heat from the outside must be transferred into the rocket. This implies that the air temperature inside the bottle must be lower than the outside air temperature. To maximize the effect of the heat exchange on the rocket's impulse, the initial air temperature inside the bottle T_0 is assumed to be $\approx 272K$, roughly the same as the outside temperature, which is the same

⁴³ Woan, *The Cambridge Handbook of Physics Formulas*, 110.

⁴⁴ Tipler and Mosca, *Physik*, 632.

as assuming the pressurisation of the rocket to be an isothermal process. By applying the formulas listed above, it is possible to approximate the temperature inside the rocket after all water was ejected.

From Eq. (3) using the default parameters:

$$p_1 = \frac{p_0 V_0^\gamma}{V_1^\gamma} \approx \frac{500000 \cdot 0.001^{1.4}}{0.0015^{1.4}} \approx 283.43 \text{ kPa}$$

By applying the ideal gas law Eq. (18):

$$\frac{p_1 V_1}{p_0 V_0} = \frac{T_1}{T_0} \quad (20)$$

$$T_1 = \frac{p_1 V_1}{p_0 V_0} \cdot T_0 = \frac{\frac{p_0 V_0^\gamma}{V_1^\gamma} V_1}{p_0 V_0} \cdot T_0 \approx \frac{283427.66681 \cdot 0.0015}{500000 \cdot 0.001} \cdot 272 \approx 231.28 \text{ K}$$

If the process is truly adiabatic, the air temperature inside the rocket could reach $\approx -40^\circ\text{C}$.

It would be very difficult to calculate the actual average air temperature inside the rocket, therefore, it is assumed to be roughly -20°C .

This allows the heat exchange rate between the inside and outside air to be approximated. Thermal conductivity for PET⁴⁵ is $k = 0.15 \text{ W/mK}$. Simplifying the bottle as a cylinder with diameter 0.1 m and height 0.2 m and using its lateral surface area leads to $A = \pi dh \approx 0.06283 \text{ m}^2$. The wall thickness Δx is assumed to be 1 mm .

$$\frac{\Delta Q}{\Delta t} = kA \frac{\Delta T}{\Delta x} \approx 0.15 \cdot 0.06283 \frac{20}{0.001} \approx 205 \text{ W}$$

A 205 W power addition to the rocket is by no means negligible, and confirms that simplifying the air expansion inside the bottle as an adiabatic process will lead to inaccurate results, but it is not enough to estimate the effect on the maximum height quantitatively. For that, a different extreme assumption will be used, specifically that the air expansion inside the rocket is an isothermal process.

$$R = 8.314462 \text{ J mol}^{-1}\text{K}^{-1} \text{ [46]}$$

From Eq. (18), it follows that:

$$n = \frac{P_0 V_0}{RT} \approx \frac{500000 \cdot 0.001}{8.314462 \cdot 272} \approx 0.221089 \text{ mol}$$

⁴⁵ Daniel Borba Marchetto et al., "A REVIEW ON POLYMER HEAT SINKS FOR ELECTRONIC COOLING APPLICATIONS," paper presented at Brazilian Congress of Thermal Sciences and Engineering, 17th Brazilian Congress of Thermal Sciences and Engineering, 2018, 2, <https://doi.org/10.26678/ABCM.ENCIT2018.CIT18-0394>.

⁴⁶ Molar Gas Constant, n.d., <https://physics.nist.gov/cgi-bin/cuu/Value?r>.

The specific heat capacity of air is given below⁴⁷:

$$C_{V(air)} \approx 20.75 \text{ J mol}^{-1} \text{ K}^{-1}$$

Work done by an adiabatic process⁴⁸:

$$W_{ad.} = nC_V \Delta T = 0.221089 \cdot 20.75 \cdot -40.72 = -186.81 \text{ J}$$

Work done by an isothermal process⁴⁹:

$$W_{isotherm.} = nRT \ln \frac{V_0}{V_1} = P_0 V_0 \cdot \ln \frac{V_0}{V_1} \approx 500000 \cdot 0.001 \cdot \ln \frac{0.001}{0.0015} = -202.73 \text{ J}$$

The isothermal expansion produces nearly 16 J more work than the adiabatic one. If those 16 J could be converted to the rocket's potential energy, the maximum height would increase by approximately 9 m. Although this is an extreme scenario, and the water rocket likely doesn't benefit this much from the heat exchange, this result, coupled with the estimation of heat exchange rate between the rocket and the air, suggests that assuming the air expansion is an adiabatic process is an oversimplification and very likely contributes to the significant difference between the rocket's maximum altitudes and the predictions of the numerical model. However, 9 m is still less than the real difference between the predicted and the measured peak of the rocket's trajectory at default parameters, therefore there are other factors involved.

One more simplification related to the adiabatic expansion is $\gamma = 1.4$, which is a value for dry air, whereas the air inside the rocket would be closer to being at 100% relative humidity. Partially, because the relative humidity of the air at the launch site was already close to 100%. To approximate the effect of this on the max. height of the rocket, the model was run with the default parameters, except γ , which was at 1.3 or 1.5. The height difference between those and the standard run with $\gamma = 1.4$ was less than 1 m.

5.3. The effect of the hydrostatic pressure:

$p_{hs} = \rho gh$ [⁵⁰]. h lies between 10 cm and 20 cm

$$p_{Max} = 1000 \cdot 9.81 \cdot 0.2 = 1962 \text{ Pa.}$$

It is less than 1% of the air pressure inside the bottle, and lies within the error of the pump, as the pump's scale division value was observed to be 5 kPa. For those reasons, excluding the hydrostatic pressure is not the simplification that led to such discrepancies.

⁴⁷ *Specific Heat Capacity of Air: Isobaric and Isochoric Heat Capacities at Various Temperatures and Pressures*, n.d., https://www.engineeringtoolbox.com/air-specific-heat-capacity-d_705.html.

⁴⁸ Tipler and Mosca, *Physik*, 589.

⁴⁹ Tipler and Mosca, *Physik*, 580.

⁵⁰ Mauri, *Transport Phenomena in Multiphase Flows*, 24.

5.4. Evaluating thrust produced by air leaving the bottle

$$p_1 \approx \frac{p_0 V_0^\gamma}{V_1^\gamma} \approx \frac{500000 \cdot 0.001^{1.4}}{0.0015^{1.4}} \approx 283.42767 \text{ kPa}$$

By expressing u from Eq. (2) and using p_1 as p :

$$u = \sqrt{\frac{2}{\rho} \cdot (p_1 - p_a)} \approx \sqrt{\frac{2}{1.27} \cdot (283 - 100) \cdot 10^3} \approx 537 \text{ m/s}$$

$$T = \rho u^2 A \approx 1.27 \cdot 537^2 \cdot 0.0003801327 \approx 140 \text{ N}$$

By using the same principles as in Eq. (16):

$$V_{\text{exited air}} = \int A \cdot u \cdot dt$$

By simplifying u to be constant:

$$V_{\text{exited air}} = t \cdot A \cdot u$$

By simplifying air expansion to be an isothermal process, the air volume can still increase by ≈ 2.83 times from the point of P_1 . Therefore:

$$V_{\text{exited air}} \approx V_1 \cdot 2.83 \approx 0.004245 \text{ m}^3$$

Then, t can be approximated:

$$t \approx \frac{V_{\text{exited air}}}{A \cdot u} \approx 0.0208 \text{ s}$$

Then, the impulse created by air is⁵¹:

$$p_{\text{air}} = T_{\text{air}} \cdot t \approx 140 \cdot 0.0208 \approx 2.9 \text{ Ns}$$

$$p_{\text{air}} = m_r \cdot v,$$

$$v = p_{\text{air}}/m_r \approx 2.9/0.181 \approx 16.023 \text{ m/s}$$

Considering that the maximum velocity of the rocket, predicted by the model at default parameters, is less than 30 m/s, an addition of 16 m/s is not negligible. The effect of the air thrust is overestimated here, due to the level of simplification in the calculation, in the assumption of u being constant and using the highest value for u , as well as assuming an isothermal expansion. Nevertheless, it shows that just like with adiabatic expansion, its insignificance must not be taken for granted.

⁵¹ Tipler and Mosca, *Physik*, 212.

5.5. The error due to the solid-body assumption

The numerical model handles the rocket during the thrust phase as a single solid body that has a mass of $m_r + m_{water}$. Therefore, according to the current model, the rocket must do work to lift the water, until the water leaves the rocket. However, in reality the work done by the rocket to lift the water inside may be much lower. As shown in Fig. 13, some water is located directly above the bottle's opening. After the rocket decouples from the launchpad, this part of the water is free to fall out of the bottle. The rocket has no efficient way of lifting the water that is directly above the nozzle.

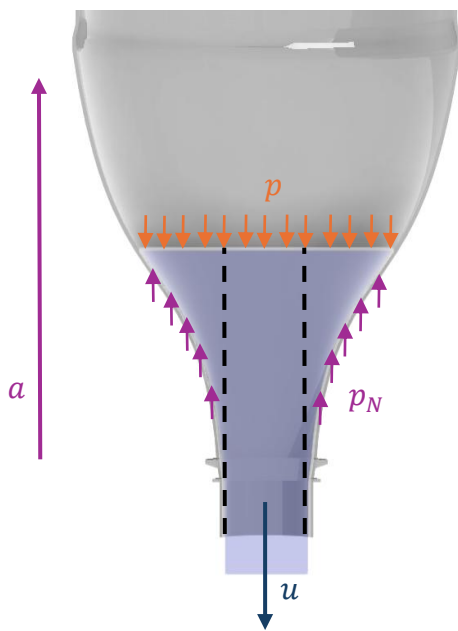


Figure 13: Pressure diagram on water

Considering the fact that all water molecules have to go through the nozzle to contribute to the reactive thrust, every water molecule is not accelerated upward by the rocket during some time. This effect means that the work done by the rocket to lift the water may be substantially lower than in the model's calculation.

Furthermore, the part of the water that is accelerated upwards by the rocket, experiences this through the pressure applied by the bottom walls of the rocket (p_N). p_N contributes to the water pressure and therefore to u .

Although not included in the data comparison chapter, the measured lift-off acceleration was much higher than the calculated by the model value. Under default parameters the rocket's lift-off acceleration was measured at 569 m/s^2 , while the model predicted 437 m/s^2 . Considering that this is the first upward acceleration detected by the on-board accelerometer, too little time has passed for heat exchange to be a major factor and the air thrust phase has not begun yet. This leads to the conclusion that the water not being lifted by the rocket and p_N contributing to u likely play the dominant role in the 132 m/s^2 acceleration difference.

Conclusion:

A numerical model of a water rocket was created. A water rocket with a launchpad and on-board altitude and acceleration logger was designed and manufactured. A total of five successful launches with varying air pressures and water volumes were conducted. Comparison between the recorded maximum altitude of the rocket and the prediction of the numerical model was performed. The model's prediction was below the recorded values for all five launches. The model yields a relative error of 70% - 100%. The comparison with another water rocket simulator suggests that inaccuracy of the model is unlikely to be caused by an unnoticed programming mistake.

The inaccuracy of the model demonstrates that high-school-level physics and mathematics are not enough to create an accurate model of a water rocket. Such inaccurate results are most likely a product of many simplifications. Thrust produced by the air leaving the rocket, water behaviour being affected by the motion of the rocket and the air expansion not being an adiabatic process are the factors which are not accounted for by the model. However, it was shown that all these factors can play a major role and should be accounted for in an accurate model.

Acknowledgments:

The author would like to thank Alexander Socher for his guidance, constructive feedback and useful advice.

The author acknowledges the support provided by "Mentoring von abschließenden Arbeiten im Bereich Physik (ABA Mentoring Physik)" program at TU Wien and would like to thank Barbara Wilfinger for her valuable advice.

The author thanks Alexander Krallinger, Robin Ferber, Sebastian Wenger and Alexander Semenov for their assistance on the launch site.

This paper's model is heavily inspired by Franco Normani's article:

<https://real-world-physics-problems.com/water-rocket-physics.html>

The mesh of a cola bottle, which is seen on some renders was made by Andy Machin and downloaded from:

<https://www.blenderkit.com/asset-gallery-detail/931d7421-e775-45e4-86f6-4f7fe66d0b05/>

Bibliography:

- “38.3 Lithium Metal and Lithium Ion Batteries.” n.d.
<https://unece.org/fileadmin/DAM/trans/danger/publi/manual/Manual%20Rev5%20Section%2038-3.pdf>.
- Adafruit ADXL375*. n.d. <https://learn.adafruit.com/adafruit-adxl375>.
- Adafruit ADXL375 Accelerometer Driver*. n.d.
https://github.com/adafruit/Adafruit_ADXL375.
- Adafruit BMP388 and BMP390 - Precision Barometric Pressure and Altimeter*. n.d.
<https://learn.adafruit.com/adafruit-bmp388-bmp390-bmp3xx/overview>.
- Adafruit LSM6DSO32 6-DoF Accelerometer and Gyroscope - STEMM A QT / Qwiic*. n.d.
<https://www.adafruit.com/product/4692>.
- Adafruit_BMP3XX*. n.d. https://github.com/adafruit/Adafruit_BMP3XX.
- Arduino IDE*. n.d. <https://www.arduino.cc/en/software/>.
- Autodesk Fusion*. n.d. <https://www.autodesk.com/eu/products/fusion-360/overview>.
- “BMP388 - Datasheet.” n.d. <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp388-ds001.pdf>.
- Borba Marchetto, Daniel, Debora Carneiro Moreira, and Gherhardt Ribatski. “A REVIEW ON POLYMER HEAT SINKS FOR ELECTRONIC COOLING APPLICATIONS.” Paper presented at Brazilian Congress of Thermal Sciences and Engineering. *17th Brazilian Congress of Thermal Sciences and Engineering*, 2018.
<https://doi.org/10.26678/ABCM.ENCIT2018.CIT18-0394>.
- Density*. n.d.
<https://unidata.github.io/MetPy/latest/api/generated/metpy.calc.density.html>.
- Department of Physics, Shivaji College (University of Delhi), Raja Garden New Delhi-110027, Ravindra Singh, Omwati Rana, et al. “Numerical Approximation Methods and Comparison with RK-4 Method for a Linear Differential Equation with Initial Conditions Using Scilab 6.1.1.” *Current Natural Sciences and Engineering* 1, no. 6 (2024): 471–79. <https://doi.org/10.63015/5c-2447.1.6>.
- Dormand, J. R., and P. J. Prince. “A Family of Embedded Runge-Kutta Formulae.” *Journal of Computational and Applied Mathematics* 6, no. 1 (1980): 19–26.
[https://doi.org/10.1016/0771-050X\(80\)90013-3](https://doi.org/10.1016/0771-050X(80)90013-3).
- Drag Equation*. n.d. <https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/drag-equation/>.

Evro, Solomon, Abdurahman Ajumobi, Darrell Mayon, and Olusegun Stanley Tomomewo. "Navigating Battery Choices: A Comparative Study of Lithium Iron Phosphate and Nickel Manganese Cobalt Battery Technologies." *Future Batteries* 4 (December 2024): 100007. <https://doi.org/10.1016/j.fub.2024.100007>.

FindFit. n.d. <https://reference.wolfram.com/language/ref/FindFit.html>.

Foma Skorokhod. *Physical Numerical Model of a Water Rocket*. n.d. <https://github.com/tom-737/Physical-numerical-model-of-a-water-rocket>.

Foma Skorokhod. *Water Rocket On-Board Altitude and Acceleration Logger*. n.d. <https://github.com/tom-737/Water-rocket-on-board-altitude-and-acceleration-logger>.

Franco Normani. *Water Rocket Physics*. n.d. <https://real-world-physics-problems.com/water-rocket-physics.html>.

GitHub. n.d. <https://github.com/>.

<https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/bernoullis-equation/>. n.d.

Kai Morich. *Serial Bluetooth Terminal*. n.d. https://play.google.com/store/apps/details?id=de.kai_morich.serial_bluetooth_terminal&hl=en_US.

LP603449-Standard Type Lithium Polymer Battery. n.d. <https://www.eemb.com/product-147>.

Matplotlib. n.d. <https://matplotlib.org/>.

Mauri, Roberto. *Transport Phenomena in Multiphase Flows*. Fluid Mechanics and Its Applications 112. Springer, 2015. <https://doi.org/10.1007/978-3-319-15793-1>.

MetPy. n.d. <https://unidata.github.io/MetPy/latest/index.html>.

Microsoft Excel. n.d. <https://excel.cloud.microsoft/en-us/>.

Mixing_ratio_from_relative_humidity. n.d. https://unidata.github.io/MetPy/latest/api/generated/metpy.calc.mixing_ratio_from_relative_humidity.html.

Molar Gas Constant. n.d. <https://physics.nist.gov/cgi-bin/cuu/Value?r>.

NumPy. n.d. <https://numpy.org/>.

Numpy.Max. n.d. <https://numpy.org/doc/stable/reference/generated/numpy.max.html>.

Open-Meteo. n.d. <https://open-meteo.com/>.

Python. V. 3.12.6. n.d. <https://www.python.org/>.

Rocket Modeler-III. n.d. <https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/rocketmodeler/>.

Shaaban, S. “Optimization of Orifice Meter’s Energy Consumption.” *Chemical Engineering Research and Design* 92, no. 6 (2014): 1005–15. <https://doi.org/10.1016/j.cherd.2013.08.022>.

Specific Heat Capacity of Air: Isobaric and Isochoric Heat Capacities at Various Temperatures and Pressures. n.d. https://www.engineeringtoolbox.com/air-specific-heat-capacity-d_705.html.

Sun, Jie, Jigang Li, Tian Zhou, et al. “Toxicity, a Serious Concern of Thermal Runaway from Commercial Li-Ion Battery.” *Nano Energy* 27 (September 2016): 313–19. <https://doi.org/10.1016/j.nanoen.2016.06.031>.

Tipler, Paul Allen, and Gene Mosca. *Physik: für Wissenschaftler und Ingenieure*. 7. dt. Aufl. Lehrbuch. Springer Spektrum, 2015.

Water Rocket Simulator. n.d. <http://www.aircommandrockets.com/sim/simulator.htm>.

Woan, Graham. *The Cambridge Handbook of Physics Formulas*. 1st ed. Cambridge University Press, 2000. <https://doi.org/10.1017/CBO9780511755828>.

Wolfram Engine. n.d. <https://www.wolfram.com/engine/>.

Yang, Yuqi, Longbin Liu, and Moyan Chen. “Study on Thrust Performance of Small Water Rocket Launch.” *Journal of Physics: Conference Series* 2313, no. 1 (2022): 012021. <https://doi.org/10.1088/1742-6596/2313/1/012021>.

Appendix 1: Notes on creating the numerical model

The process of creating the numerical model with applied simplifications is relatively straightforward. There were only a few bugs or peculiarities that needed to be resolved. The problem with $u(t)$ values after the bottle is empty, was resolved by implementing conditional checks in $u(t)$, $m(t)$, and $T(t)$ functions. Proper values were assigned after the bottle became empty. One additional issue was encountered. The air drag force was directed permanently downwards by a minus sign placed before F_D in all calculations. This has led to the rocket obtaining unrealistically high downward velocity after some time according to the model, as F_D is proportional to v^2 . An if statement had to be

added, which used a formula with $-F_D$ when the rocket flew upward and $+F_D$ when the rocket fell.

Appendix 2: Notes on creating the launchpad and the rocket

Designing and 3D printing the launchpad and rocket's parts was the most time-consuming part of this project. While it could take as little as half an hour to modify a part in CAD software (Autodesk Fusion), 3D-printing and launching the rocket to identify mistakes could take days. The rocket could only be launched before sunset for safety reasons, limiting available days. Many parts of the launchpad and of the rocket have undergone numerous changes and many versions were discarded. Especially the nose cone and the connection between the launchpad and the PET bottle faced the most redesigns. Some of the prototype 3D prints can be seen on Fig.18.

Before the measurement equipment was installed on the rocket, the dummy payload made of 1-Cent coins with a similar mass was placed and launched to see if the rocket would survive the crash with sensors on board. Using coins was convenient, as they stack well and allow to regulate the mass precisely.

Initially, it was originally planned to just print the nose cone out of PLA filament and hope that it survives the crash. This approach was unsuccessful. Three rockets with nose cones and nose cone - bottle connections of different thicknesses were launched, and all failed. Increasing the wall thickness of some parts moved and reduced cracks but did not eliminate them. A different strategy had to be used - a nose cone that can deform but absorb energy without any damage. To achieve that, a new nose cone was designed to be flexible, and to accommodate a Styrofoam part to further protect the electronics from the impact, shown in Fig.14. It was printed from TPU filament. The new nose cone was able to survive the collision with the ground, while protecting the electronics. Nevertheless, the new design had much less space to house the measurement electronics.

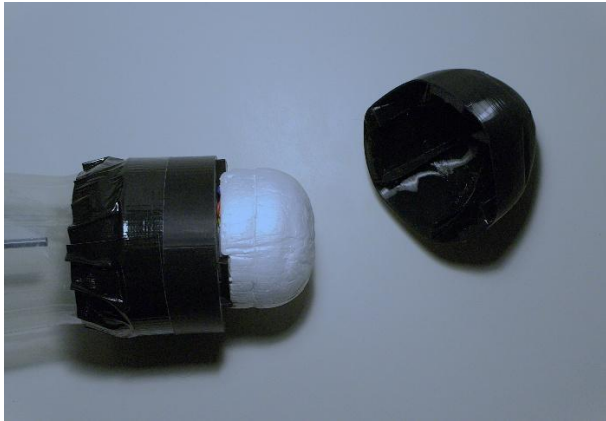


Figure 14: TPU nose cone and Styrofoam electronics protector.

Another part of the rocket that also faced some redesigns was the stabilizers. The first revision was decent, but it broke very easily and was not modular. This meant that if one fin broke, the whole structure needed to be replaced. The second version aimed at fixing that. An attempt to print stabilizers from TPU filament was also made, but the result was rejected as the efficiency of flexible fins is highly questionable, even if they are more durable.

As for the launchpad, the connector to the rocket was the most difficult part to design since it had to be watertight without adding too much friction with the bottle. The first design required the bottle to have a cap on and a nozzle in a form of a opening in the cap, can be seen in Fig.16. The bottle would have been pressed down on an O-ring making a seal around the opening. This, however, has proven to be very difficult and was ultimately abandoned. Instead, an open bottle was used, with opening acting as nozzle and as sleeve, illustrated on Fig.15. To pressurize the bottle, an insert with an O-ring was inserted into the bottle. Both versions of the coupler required many prints due to wrong tolerances to other dimensions. For the final version, a special part was designed that consisted of many plastic cylinders with different diameters, to select the optimal diameter.

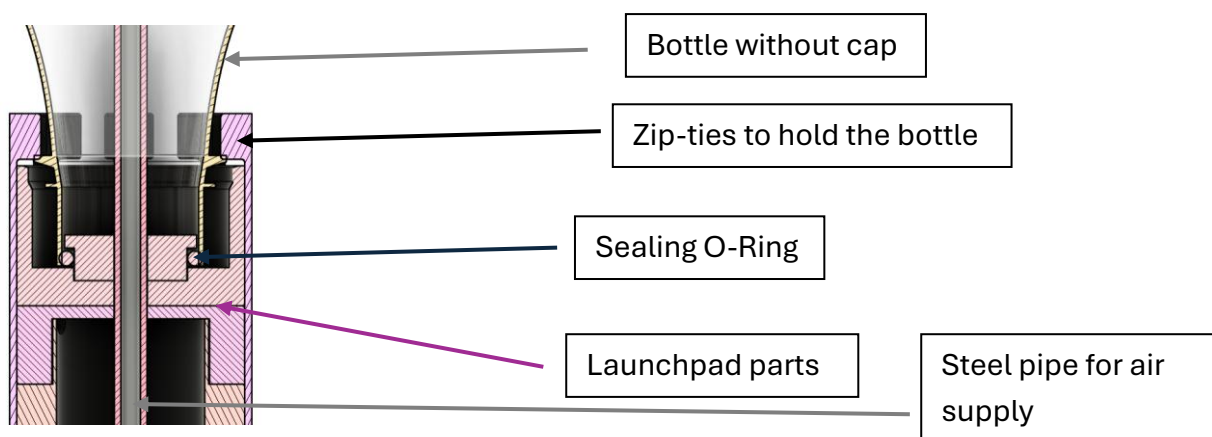


Figure 15: Final bottle - launchpad connection mechanism.

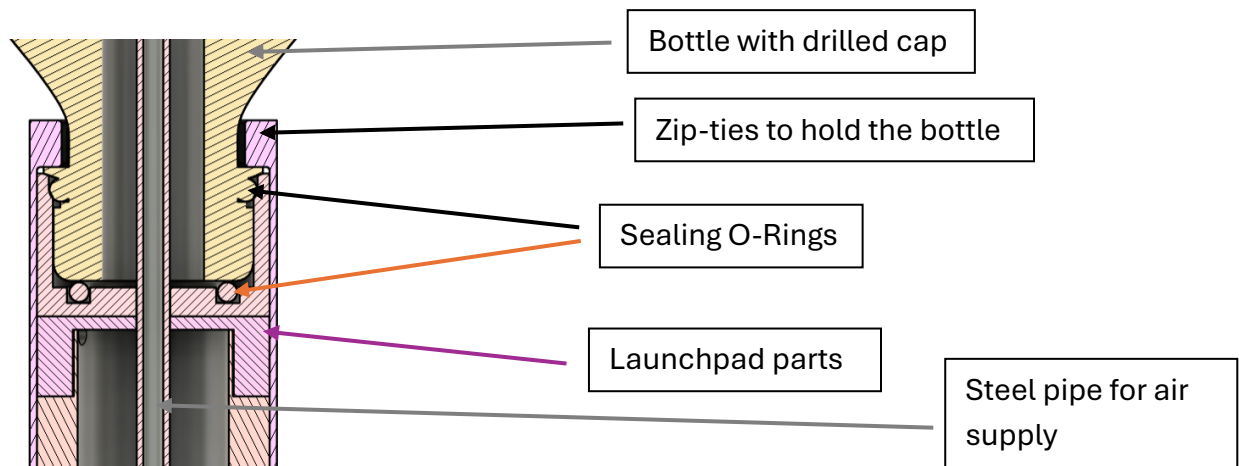


Figure 16: Initial idea of bottle-launchpad connection.

One more part, which had multiple versions, were the legs of the launchpad. The first ones were fully plastic and insufficiently rigid, shown in Fig. 17. After some rocket launches, they broke. The new ones were steel rods, held by a thick and rigid plastic part.



Figure 17: Old broken legs of the launchpad.



Figure 18: Some defective, broken and rejected parts.

Appendix 3: Notes on creating the measurement system

While there are many ways to obtain the flight data of the rocket, like using trigonometry to calculate the rocket's height from the angle of the line of sight to the ground and the distance to the launchpad, or using a video analysis to approximate the rocket's acceleration at the take-off, on-board sensors provide much more consistent data. A couple of different development boards were considered as a platform to connect sensors to. The main candidates were Arduino Nano, Raspberry Pi Pico and some ESP32 development boards. The main requirements were built-in wireless communication and input voltages, since adding a Bluetooth module or a voltage converter as separate boards would take more space and make the rocket heavier.

Ultimately, the LOLIN32 lite ESP32 development board was chosen as it fulfills all of the criteria above.

As described in the Experimental Setup chapter, a barometer and an accelerometer were used. Originally, it was planned to use two Adafruit LSM6DSO32 accelerometers and a single ADXL375 accelerometer along with a BMP388 barometer. The measurement system V1 with all of those sensors was assembled and is shown on Fig.19. LSM6DSO32 provides a range of $\pm 32\text{ g}$ ⁵² while ADXL375 provides $\pm 200\text{ g}$ ⁵³. Later, due to the drastic redesign of the rocket's nose cone, the space for the measurement system was severely limited, as can be seen in Fig.19 and Fig.20. Only two sensors could fit. ADXL375 and BMP388 were selected. The LSM6DSO32 was observed to have less variance in readings than ADXL375, but as it was calculated, the range of 32 g is not enough to cover the maximum acceleration of the rocket. Due to this, LSM6DSO32 sensors could only be used for approximating the C_D , which could also be done by BMP388. The latter was chosen instead of LSM6DSO32, because it simplified assessment of the rocket's altitude by not requiring to numerically integrate acceleration twice.

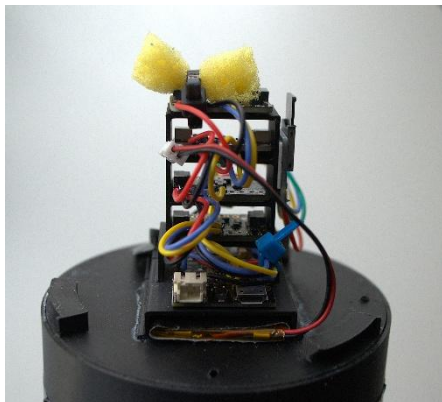


Figure 19: Measurement system V1

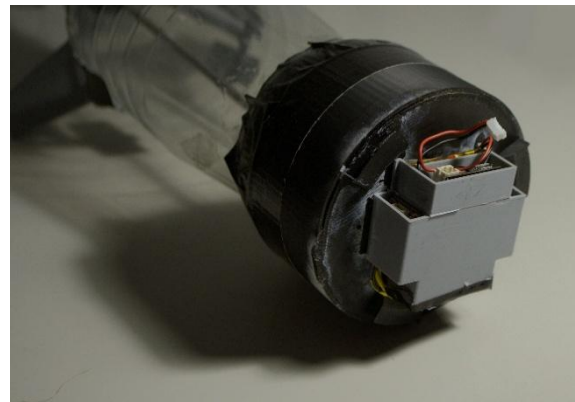


Figure 20: Measurement system V2

The SD card was used for storing the sensor readings, as the easy-to-use option.

Originally it was planned for the system to record sensor values with a sample rate of 500 Hz to plot $a(t)$ in good resolution while the rocket still has thrust, but it turned out to be unrealistic. Only 50 Hz were achieved, even after doubling the I²C speed. Possibly writing the registered values to the SD card at every iteration is a bottleneck.

Initially a Li-Ion polymer battery⁵⁴ was considered as a power source for the system, but when everything was ready for launches, it was decided to replace the battery with a

⁵² Adafruit LSM6DSO32 6-DoF Accelerometer and Gyroscope - STEMMMA QT / Qwiic, n.d., <https://www.adafruit.com/product/4692>.

⁵³ Adafruit ADXL375.

⁵⁴ LP603449-Standard Type Lithium Polymer Battery, n.d., <https://www.eemb.com/product-147>.

LiFePO₄ one. The battery type and size are the only differences between V2 and V3. The measurement system V3 on Fig.4 has a LiFePO₄ battery installed. The acceleration of the rocket during impact with the ground exceeds the certified safe values for LiPo batteries⁵⁵ and damaged LiPo batteries can cause a fire or an explosion⁵⁶ as well as emit toxic gases⁵⁷. LiFePO₄ batteries, on the other hand, are known for their safety⁵⁸.

The data logger for the ESP32 board was programmed in Arduino IDE. A Bluetooth console relying on a free Android app - Serial Bluetooth Terminal⁵⁹ was implemented with the following commands: **sensors** - display current sensor values, **ls** - list all files on the SD card and **arm** - start logging. Although not strictly necessary, the flight loggers' software takes advantage of ESP32's multitasking capabilities.

After sending the arm command, the flight logger asks for a filename and recording duration. Furthermore, a calibration function is initiated, reading accelerometer values to offset the sensor, so that it outputs *g* in equilibrium. For barometer, the ground pressure is saved, as it is required to calculate the altitude from pressure using BMP388 library's built-in functions.

Then, the flight logger starts reading sensors in a loop, which ends when an upward acceleration is detected. Since this moment, every accelerometer and barometer value with a corresponding timestamp is recorded onto SD card in a .csv file.

⁵⁵ "38.3 Lithium Metal and Lithium Ion Batteries," n.d., <https://unece.org/fileadmin/DAM/trans/danger/publi/manual/Manual%20Rev5%20Section%2038-3.pdf>.

⁵⁶ Stated on the packaging "Risk of fire, explosion or burning"

⁵⁷ Jie Sun et al., "Toxicity, a Serious Concern of Thermal Runaway from Commercial Li-Ion Battery," *Nano Energy* 27 (September 2016): 313–19, <https://doi.org/10.1016/j.nanoen.2016.06.031>.

⁵⁸ Solomon Evro et al., "Navigating Battery Choices: A Comparative Study of Lithium Iron Phosphate and Nickel Manganese Cobalt Battery Technologies," *Future Batteries* 4 (December 2024): 1, <https://doi.org/10.1016/j.fub.2024.100007>.

⁵⁹ Kai Morich, *Serial Bluetooth Terminal*, n.d., https://play.google.com/store/apps/details?id=de.kai_morich.serial_bluetooth_terminal&hl=en_US.